5.6 End-to-End Tests

Theoretically, testing frameworks such as Vitest enable you to cover all levels of the test pyramid. This class of frameworks is best suited for the formulation of unit and integration tests. There are separate frameworks for end-to-end tests, such as Selenium, Cypress, or Playwright, which have specialized in making the implementation of end-to-end tests as convenient as possible for you.

In the following sections, you'll learn how to use Playwright to generate an end-to-end test for a simple login form. The form has an input field for the username and one for the password. There is also a button for sending the form. The form is transmitted via a JavaScript function. Upon successful login, users are redirected to the `/list` path. The HTML code of the form reads as follows:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <title>Login Form</title>
  </head>
  <body>
    <div class="login-container">
      <h2>Login</h2>
      <form id="loginForm">
        <input
          type="text"
          id="username"
          name="username"
          placeholder="Username"
```

```
            required
          />
          <input
            type="password"
            id="password"
            name="password"
            placeholder="Password"
            required
          />
          <button type="submit">Login</button>
        </form>
      </div>
    </body>
  </html>
```

We deliberately omit the styling and JavaScript here because the end-to-end test is a classic black box test that checks the functionality from the user's perspective.

5.6.1 Generating an End-to-End Test

The prerequisite for your end-to-end test to work is an installation of Playwright, which you can achieve via the `npx init playwright@latest` command. You can run the Playwright tests using the `npx playwright test` command. However, only sample tests are available immediately after installation. But you can change this by asking ChatGPT to generate a test for the login form for you.

For the test to work, the AI tool needs some information from you. You must provide the HTML structure and specify the address via which the form can be accessed. You must also specify which user name and password should be used and where you'll be redirected to after successful login. The corresponding prompt could look as follows:

Prompt: Create a Playwright test for the following HTML structure. Assume there is JavaScript that handles the form submission. The form can be found at *http://localhost:3000/login*. Use "admin" as username and "test" as password. After the successful login, the user will be redirected to *http://localhost:3000/list*.

[HTML form code]

Here is a Playwright test for the provided HTML structure. This test script will fill in the username and password fields with “admin” and “test” respectively, submit the form, and check if the user is redirected to http://localhost:3000/list.

First, make sure you have Playwright installed. If not, you can install it using:

```
npm install @playwright/test
```

Next, create a test file (e.g., login.test.js) and add the following code:

```
// @playwright/test module
import { test, expect } from "@playwright/test";

test("login test", async ({ page }) => {
  // Go to the login page
  await page.goto("http://localhost:3000/login");

  // Fill in the username
  await page.fill("#username", "admin");

  // Fill in the password
  await page.fill("#password", "test");

  // Click the login button
  await page.click('button[type="submit"]');

  // Check if the the redirected URL is the list page
  await page.waitForURL("http://localhost:3000/list");

  // Check that the new URL is the list page

expect(page.url()).toBe("http://localhost:3000/list");
});
```

[explanation of the code]

Create a new file named `login.test.js` in the `test` directory of your application, and then run the test command as suggested in the response to the prompt. If the test runs successfully, it checks your login process. ChatGPT has also provided the individual lines with comments so you can easily follow the steps. You can find out even more about what the individual commands mean in the following explanation.

One limitation that you have to live with, at least with LLMs such as ChatGPT, is that you can't cover an entire workflow in one step with an end-to-end test. The problem here is that the AI tool can't dynamically run through a workflow that consists of multiple views. For this reason, you need to proceed step by step and then put the individual parts together to form an overall workflow.

5.6.2 Creating a Page Object

Your tests will be even clearer if you work with page objects. A *page object* provides you with speaking methods that enable you to interact with a page. The methods can have both read and write access to the page and, for example, return a specific value for checking or write values to a form field. This makes the test code even easier to read.

To use a page object in your end-to-end test, use the previous ChatGPT session, and specify the following as the next prompt:

Prompt: Could you include a page object in the test?

> Certainly! Using the Page Object Model (POM) helps to organize and maintain your test code more efficiently. Below is an example that includes a page object for the login page and incorporates it into the test.
>
> First, create a new file named `LoginPage.js` for the page object.

```
// LoginPage.js
export class LoginPage {
  /**
   * @param {import('@playwright/test').Page} page
   */
  constructor(page) {
```

```
    this.page = page;
    this.usernameInput = '#username';
    this.passwordInput = '#password';
    this.submitButton = 'button[type="submit"]';
  }
  async goto() {
    await
this.page.goto('http://localhost:3000/login');
  }
  async login(username, password) {
    await this.page.fill(this.usernameInput,
username);
    await this.page.fill(this.passwordInput,
password);
    await this.page.click(this.submitButton);
  }
}
```

Next, update the test file to use this page object.

```
// login.test.js
import { test, expect } from '@playwright/test';
import { LoginPage } from './LoginPage';

test('login test', async ({ page }) => {
  const loginPage = new LoginPage(page);

  // Navigate to the login page
  await loginPage.goto();

  // Perform the login action
  await loginPage.login('admin', 'test');

  // Check if the the redirected URL is the list page
  await page.waitForURL("http://localhost:3000/list");

  // Check that the new URL is the list page

expect(page.url()).toBe('http://localhost:3000/list');
});
```

[explanation of the code]

If it's purely a question of the number of lines, you've gained nothing by using the page object. The three lines that are lost by filling out the form are lost again by importing, instantiating the page object, and calling the methods. In a larger test, however, the code is usually much more compact and readable. You can also reuse the page objects between different end-to-end tests and gain from this. For example, if you swap out the login process to such a page object, you can use it in every end-to-end test in which you have to log in.